

Nature Finder 🌿

Nature Finder est un jeu d'exploration dans lequel vous incarnez Lauren, un chercheur qui part à la découverte de la forêt du Limousin. À travers votre exploration, vous trouverez des animaux et végétaux originaux d'ici. Complétez votre journal de recherche avec des cartes à collectionner que vous obtiendrez en interagissant avec eux.



Cahier des charges

L'objectif de ce projet est de créer un jeu en Python avec PyGame, une librairie conçue pour développer des jeux. Le thème du jeu que nous devons créer est "Nature", alors nous avons imaginé un jeu qui correspond à ce thème. Pour cela, nous avons dû programmer plusieurs systèmes et fonctionnalités fondamentales.

On ne verra dans le cahier des charges que les programmes principaux.

1. Le joueur

Le code du joueur se trouve dans les scripts `player.py`, qui crée la classe `Player`, et `main.py`, qui gère ses mouvements.

Pour créer le joueur, nous avons besoin de définir plusieurs paramètres dans sa classe :

- la position : où est-ce que le joueur se trouve actuellement ?
- la boîte de collision : sur quelle surface le joueur peut-il entrer en collision avec son environnement ?
- la direction : vers où est-ce que le joueur se déplace ?
- les sprites : un dictionnaire d'images du joueur en fonction de la direction dans laquelle il va

- l'inventaire : quels sont les objets que le joueur a récupérés durant son exploration ?

Avant de manipuler le joueur dans le script principal, on doit créer une instance. Cette instance représente le joueur existant dans le monde, qui possède tous les paramètres cités précédemment, et que l'on peut modifier. On met en paramètres la position X,Y initiale du joueur, définie par des valeurs constantes.

```
START_X = 5 * TILE_SIZE # 5 cases vers la droite
START_Y = 6 * TILE_SIZE # 6 cases vers le bas
player = Player(START_X, START_Y)
```

Pour permettre au joueur de se déplacer, on récupère d'abord la liste des touches du clavier pressées par le joueur sur cette frame, puis on calcule la direction X,Y à lui appliquer en fonction des touches pressées (flèches gauche, droite, haut, bas).

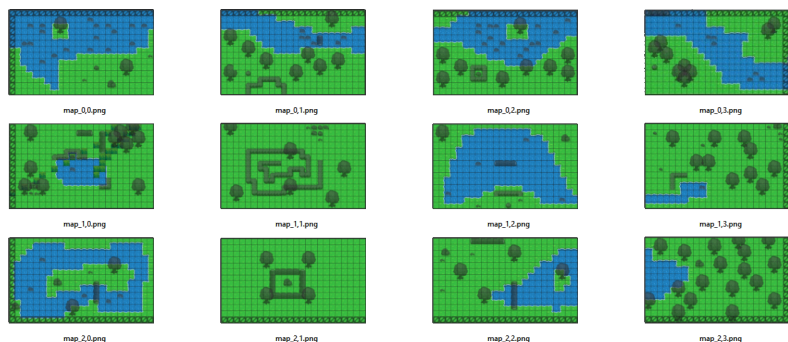
```
keys = pygame.key.get_pressed()
dx = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
dy = keys[pygame.K_DOWN] - keys[pygame.K_UP]
```

Puis, on appelle la fonction `move()` du joueur avec les paramètres `dx` et `dy`.

```
player.move(dx, dy, walls)
```

2. [Le monde du jeu](#)

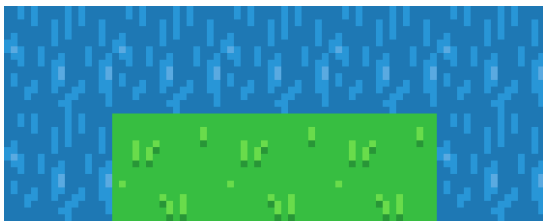
La carte (ou map) du jeu est composée de 12 scènes reliées entre elles ; ainsi le joueur peut se balader entre les différentes scènes en s'approchant d'un côté de la map.



Les maps font partie d'un dictionnaire `MAPS_DATA`, où les clés sont la position de la map sur une grille de 4x3, et les valeurs sont un Array des Tiles (ou images) présentes sur cette map.

Voici un exemple d'une map écrite dans le code du jeu :

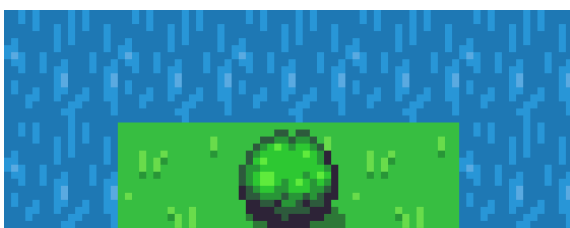
```
(0, 0) : [  
  ["water"], ["water"], ["water"], ["water"], ["water"]], # Ligne  
1  
  ["water"], ["grass"], ["grass"], ["grass"], ["water"]], # Ligne  
2  
]
```



Maintenant, imaginons que nous avons besoin de placer un buisson sur l'herbe. On pourrait essayer de remplacer "grass" par "bush", mais le buisson apparaîtrait seul, sans herbe en-dessous de lui. C'est pour cela que chaque case est elle-même représentée par un Array, où le premier élément est celui affiché dessous, et le dernier est affiché au-dessus.

Exemple :

```
(0, 0) : [  
  ["water"], ["water"], ["water"], ["water"], ["water"]], # Ligne  
1  
  [{"w"}, {"grass"}, {"grass", "bush"}, {"grass"}, {"w"}], # Ligne  
2  
]
```



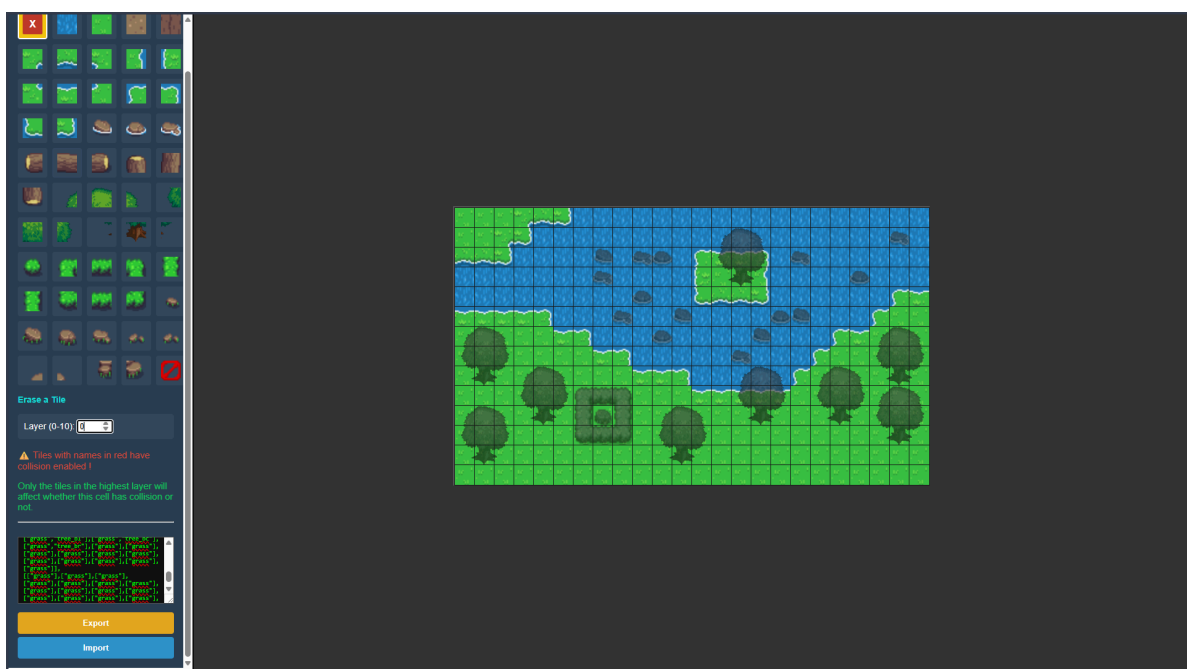
Enfin, certains éléments de décors ont besoin d'apparaître au-dessus du joueur. C'est pour cela que nous avons rajouté une règle d'affichage du jeu : les Tiles en dessous du joueur (couches 0 et 1) sont ajoutées à une liste `tiles_below_player`, et les autres Tiles (couches 2 à 10) sont ajoutées à la liste `tiles_above_player`. À l'affichage du jeu, on dessine d'abord les Tiles en dessous du joueur, puis le joueur lui-même, enfin les Tiles au-dessus.

```
for img, pos in tiles_below_player: # Tiles en dessous
    screen.blit(img, pos)

if(player): # Joueur
    player.draw(screen)

for img, pos in tiles_above_player: # Tiles au-dessus
    screen.blit(img, pos)
```

Après avoir développé toutes ces fonctionnalités pour la map du jeu, nous nous sommes rendu compte de la complexité de devoir créer une map : il faudrait, à la main, écrire l'identifiant des Tiles dans des Arrays, pour les 336 cases de chaque map (car une map fait 14 cases de long et 24 de haut). Ce serait un travail trop fastidieux. C'est pour cela que nous avons développé une application Web permettant de placer des Tiles dans des cases, et avec la couche souhaitée.



3. Les objets (interactables)

Les interactables (comme appelés dans le code) sont des objets avec lesquels le joueur peut interagir. Ils sont, le plus souvent, utilisés comme animaux ou végétaux que le joueur peut ramasser afin de compléter sa collection. Ils ont leur propre classe dans le script `interactable.py`, et possèdent ces propriétés :

- l'image de l'objet
- l'identifiant (doit être unique)
- le nom
- la description (valable seulement pour les objets pouvant être récupérés)

Le joueur peut interagir avec eux en appuyant sur E : cela va appeler la fonction `check_interaction()` de la classe `Player`. Cette fonction a pour rôle de savoir quoi faire en fonction de l'objet avec lequel le joueur a interagi. Si c'est un objet à collecter, alors l'objet en question va être ajouté à l'inventaire du joueur :

```
self.inventory.append(obj)
```

D'autres actions sont effectuées en fonction de l'objet.

Les Interactables à faire apparaître dans le jeu sont déterminés et configurés dans un dictionnaire `INTERACTABLES_DATA`, un peu semblable à celui des Maps. En effet, la clé est la position de la map sur une grille de 4x3, et la valeur est un Array des objets que l'on veut disposer sur cette map.

```
INTERACTABLES_DATA = {  
    # Format : (map_x, map_y): [(x, y, "id", "sprite.png", "name",  
    "description")]  
    (0, 0): [  
        [500, 500, "ours_brun", "bear.png", "Bear", "It's a  
bear"],  
        [500, 500, "mesange_nonnette", "bird1.png", "Bird1", "A  
bird"],
```

```

],
(1, 0): [
    [500, 500, "mesange_charbonniere", "bird2.png", "Another
bird"],
    [500, 500, "loup_gris", "wolf.png", "Wolf", "It's a
wolf"],
]
}

```

Algorithmes

Dans cette partie sur les algorithmes, nous allons observer et expliquer deux morceaux de programme de notre jeu. Le premier est la fonction `load_map()`, qui permet de charger la map (ses Tiles et Interactables) dans laquelle se trouve le joueur. Le système de transition dans la boucle “while” principale qui permet de transitionner entre deux maps.

1. La fonction `load_map()`

On commence par créer 3 listes vides : `walls`, `tiles_below_player` et `tiles_above_player`. Puis, on assigne à la variable `data` les données de la map actuelle : on cherche dans le dictionnaire `MAPS_DATA`, les données correspondantes aux coordonnées de la map dans laquelle le joueur se trouve (`map_coords`). Ensuite, on entre dans une première boucle `for`, qui parcourt la liste énumérée `data`. Ici, `enumerate()` permet d’obtenir un compteur à la liste et d’attribuer un index à chaque élément. Cela permet de savoir quelle ligne de la map le code est en train d’utiliser. Puis, on entre dans une autre boucle `for`, qui cette fois-ci parcourt la ligne actuelle. Grâce à ces deux boucles, on obtient les coordonnées X,Y, en pixels, de la case que le code est en train de traiter. On entre dans une dernière boucle `for`, qui parcourt toutes les Tiles placées sur cette case (car il peut y avoir autant de Tiles sur une case qu’il n’y a de couches). On attribue ensuite à `tile_info`, les informations de la Tile qu’on regarde dans la boucle `for`. Si la couche (obtenue à partir de l’index d’énumération) est inférieur à 2, on l’ajoute à `tiles_below_player`. Sinon, on l’ajoute à `tiles_above_player`. La dernière chose à faire pour nos Tiles est d’assigner la Tile la plus haute

de la case à `highest_tile_info`. Enfin, si cette Tile possède de la collision, on l'ajoute à `walls`.

On passe maintenant au chargement des Interactables, qui lui est plus facile à comprendre. D'abord, on crée une liste `interactables`. On obtient tous les interactables de la map en les mettant dans la liste `objs`, puis on parcourt cette liste avec la boucle `for`, ce qui nous permet de récupérer la position, l'identifiant, le sprite, le nom et la description de chaque objet. Enfin, on ajoute l'objet à la liste `interactables`, et on retourne nos 4 listes.

2. Le système de transition

On commence par créer une variable booléenne `transition`. Ensuite, on crée 4 blocs `if`, qui ont tous le même rôle, mais ne fonctionnent pas avec les mêmes variables. Le premier s'exécute si le joueur sort de la map par la droite, le deuxième par la gauche, le troisième par le bas, et le quatrième par le haut. En fonction de là où le joueur sort, on le fait réapparaître à l'opposé de la map, et on met `transition` à `True`. Un autre bloc `if` exécuté seulement quand `transition` est à `True` continue le script. À l'intérieur, on y trouve deux actions à effectuer : la première est une mesure de sécurité qui empêche le joueur de sortir de la map. Par exemple, si le joueur sort de la map 2,3 par la droite, il ne se retrouvera pas dans le vide, mais réapparaîtra juste de l'autre côté de la map dans laquelle il se trouve déjà. La deuxième action est la même qui est réalisée au lancement du jeu : elle permet de remplir les 4 listes `tiles_below_player`, `tiles_above_player`, `walls` et `interactables`.

Carnet de bord

Vous trouverez ici, sous forme de tableau, tout ce que nous avons fait pour réaliser ce projet.

1. Gabriel

Date	Activités réalisées	Durée
Samedi 03 Janvier	Composition de la musique du jeu et enregistrement. Mixage audio sur <i>LMMS</i> .	5 heures
Dimanche 04 Janvier	Recherche et conception sur l'identité graphique et design de la map.	3 heures
Samedi 10 Janvier	Recherche de textures et création des 10 premières cartes à collectionner.	8 heures
Dimanche 11 Janvier	Finalisation des 3 cartes et construction de la map (level design).	4 heures
Samedi 17 Janvier	Finalisation de la map.	4 heures

2. Charles

Date	Activités réalisées	Durée
Samedi 03 Janvier	Création de la base du jeu et des mouvements du joueur.	2 heures
Dimanche 04 Janvier	Développement de la première version du système de map.	3 heures
Mercredi 07 Janvier	Création de l'application Web pour faciliter la construction de la map.	5 heures

Samedi 10 Janvier	Programmation du système de transition.	1 heure
Dimanche 11 Janvier	Création de la classe Player.	2 heures
Mercredi 14 Janvier	Écriture du système d'Interactables.	4 heures
Samedi 17 Janvier	Création de l'interface de l'inventaire.	3 heures
Dimanche 18 Janvier	Finalisation globale du jeu.	3 heures

main.py

```
import pygame
from settings import *
from utils import load_tiles_config
from player import Player
from interactable import Interactable
from inventory_ui import InventoryUI

pygame.init()
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
clock = pygame.time.Clock()

pygame.mixer.init() # Initialise le système audio

try:
    pygame.mixer.music.load("audio/music.mp3") # Charge le fichier
    audio
    pygame.mixer.music.play(loops=-1) # Lance la lecture en boucle
    pygame.mixer.music.set_volume(1) # Règle le volume
except pygame.error as e:
    print(f"Cannot load music : {e}")

TILE_CONFIG = load_tiles_config("tiles.json", TILE_SIZE) # Charge
les données des Tiles

def load_map(map_coords):
    walls = []
    tiles_below_player = []
    tiles_above_player = []

    data = MAPS_DATA.get(tuple(map_coords), MAPS_DATA[(0, 0)]) #
    Récupère les données de la carte

    for r, row in enumerate(data):
        for c, tile_stack in enumerate(row):
            x, y = c * TILE_SIZE, r * TILE_SIZE # Calcule la
            position en pixels
            highest_tile_info = None

            for i, tile_id in enumerate(tile_stack):
                if not tile_id:
                    continue

                tile_info = TILE_CONFIG.get(tile_id)
                if tile_info:
                    if i < 2: # Assigne aux couches inférieures
```

```

tiles_below_player.append((tile_info["image"], (x, y)))
                        else: # Assigne aux couches supérieures

tiles_above_player.append((tile_info["image"], (x, y)))

                        highest_tile_info = tile_info

                        if highest_tile_info and
highest_tile_info["collision"]: # Si la Tile la plus haute de
cette case a de la collision
                        walls.append(pygame.Rect(x, y, TILE_SIZE,
TILE_SIZE)) # On la met dans la liste "walls"

                        interactables = []
                        objs = INTERACTABLES_DATA.get(tuple(map_coords), [])
                        for x, y, uid, sprite, name in objs: # Instancie les objets de
la map
                                interactables.append(Interactable(x, y, uid, sprite,
name))

                        return tiles_below_player, tiles_above_player, walls,
interactables

START_MAP = [0, 0]
START_X = 2 * TILE_SIZE
START_Y = 13 * TILE_SIZE

current_map_pos = START_MAP

tiles_below_player, tiles_above_player, walls, interactables =
load_map(current_map_pos) # Initialise l'environnement

player = Player(START_X, START_Y) # Initialise le joueur
inventory_ui = InventoryUI() # Initialise l'interface utilisateur

running = True
while running:

    for event in pygame.event.get(): # Boucle des événements
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            if event.key == pygame.K_e: # Déclenche la détection
d'interaction

```

```

        collected_item =
player.check_interaction(interactables)
        if event.key == pygame.K_a: # Alterne l'état de
l'affichage UI
            inventory_ui.toggle()

        if event.type == pygame.MOUSEBUTTONDOWN and event.button
== 1:
            if inventory_ui.is_open: # Transmet les coordonnées du
clic à l'UI
                inventory_ui.handle_click(event.pos,
player.inventory)

        keys = pygame.key.get_pressed()
        dx = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        dy = keys[pygame.K_DOWN] - keys[pygame.K_UP]

        if not inventory_ui.is_open: # Bloque le mouvement si
l'inventaire est ouvert
            player.move(dx, dy, walls)

        transition = False # Variable booléenne pour le changement de
map

        if player.rect.left > SCREEN_WIDTH:
            current_map_pos[0] += 1
            player.pos.x = -5
            transition = True
        elif player.rect.right < 0:
            current_map_pos[0] -= 1
            player.pos.x = SCREEN_WIDTH-5
            transition = True
        elif player.rect.top > SCREEN_HEIGHT:
            current_map_pos[1] += 1
            player.pos.y = -5
            transition = True
        elif player.rect.bottom < 0:
            current_map_pos[1] -= 1
            player.pos.y = SCREEN_HEIGHT-5
            transition = True

        if transition: # Actualise les données lors du changement de
map
            current_map_pos[0] = max(0, min(3, current_map_pos[0]))
            current_map_pos[1] = max(0, min(2, current_map_pos[1]))
            tiles_below_player, tiles_above_player, walls,
interactables = load_map(current_map_pos)

```

```

        player.rect.topleft = (int(player.pos.x),
int(player.pos.y))

    screen.fill((0, 0, 0)) # Efface l'écran

    for img, pos in tiles_below_player: # Affiche l'arrière-plan
        screen.blit(img, pos)

    for obj in interactables: # Affiche les objets Interactables
        obj.draw(screen)

    if(player): # Affiche le joueur
        player.draw(screen)

    for img, pos in tiles_above_player: # Affiche le premier plan
        screen.blit(img, pos)

    inventory_ui.draw(screen, player.inventory) # Affiche
l'interface

    pygame.display.flip()
    clock.tick(60) # Bloque la fréquence de rafraichissement à 60
FPS

pygame.quit()

```

player.py

```

import pygame
from utils import load_img
from settings import PLAYER_DISPLAY_SIZE,
PLAYER_INTERACTION_AREA_SIZE, SPEED

class Player:
    def __init__(self, x, y):
        self.pos = pygame.Vector2(x, y) # Position du joueur

        self.hitbox_size = (35, 35) # Dimensions de la boîte de
collision
        self.hitbox_offset = pygame.Vector2(0, 25) # Décalage pour
aligner la collision aux pieds

        self.rect = pygame.Rect(0, 0, *self.hitbox_size) #
Initialisation de la hitbox
        self.update_hitbox_pos() # Position initiale de la hitbox

        self.current_dir = (0, 1) # Direction par défaut vers le
bas

```

```

        self.sprites = { # Mappage des images selon les vecteurs
de direction
            (0, -1): load_img("player_up.png",
PLAYER_DISPLAY_SIZE),
            (0, 1): load_img("player_down.png",
PLAYER_DISPLAY_SIZE),
            (-1, 0): load_img("player_left.png",
PLAYER_DISPLAY_SIZE),
            (1, 0): load_img("player_right.png",
PLAYER_DISPLAY_SIZE),
            (-1, -1): load_img("player_up_left.png",
PLAYER_DISPLAY_SIZE),
            (1, -1): load_img("player_up_right.png",
PLAYER_DISPLAY_SIZE),
            (-1, 1): load_img("player_down_left.png",
PLAYER_DISPLAY_SIZE),
            (1, 1): load_img("player_down_right.png",
PLAYER_DISPLAY_SIZE),
            (0, 0): load_img("player_idle.png",
PLAYER_DISPLAY_SIZE),
        }

        self.inventory = [] # Liste des objets récupérés

    def move(self, dx, dy, walls):
        if dx != 0 or dy != 0:
            self.current_dir = (dx, dy) # Mise à jour de la
direction
            move_vec = pygame.Vector2(dx, dy).normalize() * SPEED
# Normalisation de la vitesse en diagonale

            # Gestion des déplacements et collisions sur l'axe X
            self.pos.x += move_vec.x
            self.update_hitbox_pos()
            for wall in walls:
                if self.rect.colliderect(wall):
                    if move_vec.x > 0: self.rect.right = wall.left
                    if move_vec.x < 0: self.rect.left = wall.right
                    self.pos.x = self.rect.centerx -
self.hitbox_offset.x

            # Gestion des déplacements et collisions sur l'axe Y
            self.pos.y += move_vec.y
            self.update_hitbox_pos()
            for wall in walls:
                if self.rect.colliderect(wall):
                    if move_vec.y > 0: self.rect.bottom = wall.top

```

```

        if move_vec.y < 0: self.rect.top = wall.bottom
        self.pos.y = self.rect.centery -
self.hitbox_offset.y

        self.update_hitbox_pos() # Synchronisation de la
hitbox

    def check_interaction(self, interactables):
        # Étend la zone d'interaction autour du joueur
        interaction_rect =
self.rect.inflate(PYER_INTERACTION_AREA_SIZE,
PYER_INTERACTION_AREA_SIZE)
        for obj in interactables:
            # Vérifie la proximité et l'absence de l'objet dans
l'inventaire
            if interaction_rect.colliderect(obj.rect) and obj.id
not in [item.id for item in self.inventory]:

                if obj.id == "boat": # Condition spécifique pour
l'objet boat
                    pass
                else:
                    self.inventory.append(obj) # Ajout à
l'inventaire du joueur

                    return obj # Retourne l'objet avec lequel
l'interaction a été faite
                    return None

    def update_hitbox_pos(self):
        # Aligne le centre de la hitbox sur la position du joueur
avec son décalage
        self.rect.center = self.pos + self.hitbox_offset

    def draw(self, screen):
        # Récupère le sprite correspondant à la direction
        sprite = self.sprites.get(self.current_dir,
self.sprites[(0, 0)])

        # Centre l'image sur la position du joueur
        sprite_rect = sprite.get_rect(center=self.pos)
        screen.blit(sprite, sprite_rect)

```